26CA1103E - LINUX SYSTEMS PROGRAMMING

MINI PROJECT REPORT

ON

LIBRARY MANAGEMENT SYSTEM



Team Name: IQOO

Submitted by

S.No	Student Name	Roll No.
1	SAHIL KUMAR	2610520029
2	GUNTI SAI KARTHIK	2610520028
3	DOKUR YASHWANTH REDDY	2610520005
4	SARAH SHARMA	2610520036

Department	CSA – BCA
Year / Semester	I Year – I Semester
Course	LINUX SYSTEMS PROGRAMMING
Course Code	26CA1103E
Environment	Ubuntu Linux / WSL
Technology	Bash Shell Scripting
Course Coordinator	Dr. Alampally Sreedevi
Academic Year	2026–2027

CERTIFICATE

This is to certify that the mini project titled **"Library Management system"** has been successfully completed by **SAHIL KUMAR (R.No: 2610520029)** and **GUNTI SAI KARTHIK (R.No: 2610520028)** and **DOKUR YASHWANTH REDDY (R.No: 2610520005)** and **SARAH SHARMA (R.No: 2610520036)** of Team **IQOO** I Year – I Semester, Department of CSA – BCA, KL University, as part of the **26CA1103E -LINUX SYSTEMS PROGRAMMING** course during the academic year 2026–2027.

The project demonstrates the use of Linux command-line tools and Bash shell scripting to solve a simple academic problem

.

Dr. Alampally Sreedevi **Course Coordinator**	**Dr. D. S. Rao** **HOD- CSA-BCA**

DECLARATION

We hereby declare that the project report titled **"Library Management system"** is our own work, carried out as part of the 26CA1103E -LINUX SYSTEMS PROGRAMMING course in the Department of CSA – BCA, KL University. We have used Bash shell scripting and standard Linux commands to design and implement the project. The project has been prepared for academic learning and demonstration purposes.

SAHIL KUMAR (R.No: 2610520029)	**GUNTI SAI KARTHIK** (R.No: 2610520028)
DOKUR YASHWANTH REDDY (R.No: 2610520005)	**SARAH SHARMA** (R.No: 2610520036)

Date: 13/10/2026
Place: Hyderabad

ACKNOWLEDGEMENT

We express our sincere gratitude to our Course Coordinator Dr. Alampally Sreedevi, Department of CSA – BCA, KL University, for the guidance and encouragement given throughout this mini project.

We also thank the Head of the Department **Dr. D.S.Rao**, and our institution for providing the Linux / WSL environment required for implementation.

This project helped us understand basic Linux commands, Bash scripting, variables, conditions, arithmetic operations, functions, input/output, and file handling. Finally, we thank our friends and family for their support.

SAHIL KUMAR - (R.No: 2610520029)

GUNTI SAI KARTHIK - (R.No: 2610520028)

DOKUR YASHWANTH REDDY - (R.No: 2610520005)

SARAH SHARMA - (R.No: 2610520036)

ABSTRACT

The Library Management System is a beginner-friendly Linux Bash project designed to manage a small collection of books through an interactive terminal menu. The application allows users to add a book record, display saved books, search for books by ID, calculate a fine based on the number of late days, and exit the program. Book records are stored in a plain-text file so that data remains persistent after the script closes, operating at a fixed fine rate of ₹5 per late day without automatically tracking issue or return dates.

The project runs entirely from the Linux terminal and does not require a graphical interface or external database. It introduces students to practical shell scripting concepts such as variables, read commands, file handling, arithmetic expressions, loop control, if-else statements, functions, and formatted output.

Keywords: *Linux, Bash, Shell Scripting, Library Management, Book Records, Text File Database, Fine Calculation, Terminal*

TABLE OF CONTENTS

1. Introduction 1
2. Problem Statement 1
3. Objectives 1
4. Scope of the Project 2
5. Existing System 2
6. Proposed System 2
7. Requirements 3
8. System Design 3
9. Algorithm 4
10. Implementation 4
11. Source Code 5
12. Sample Execution 6
13. Testing 7
14. Team Member Responsibilities 7
15. Advantages and Limitations 8
16. Future Enhancements 8
17. Learning Outcomes 8
18. Conclusion 9
19. References 9
Appendix A – Basic Commands Used 9
Appendix B – Viva Questions 10

1. INTRODUCTION

Linux provides a powerful command-line environment in which users can automate repetitive tasks using shell scripts. Bash is one of the most widely used shells in Linux. For beginners, a small academic application is a useful way to understand how commands and programming logic work together.

This project uses Bash to create a simple **Library Management System**. Users can add book details, view the saved collection, search for a book by ID, and calculate a late-return fine at a fixed rate. Book records are stored in a plain-text file to ensure data persistence after the script closes.

2. PROBLEM STATEMENT

In a small library, book information may be stored in notebooks or separate files. Finding a particular book can take time, and manual fine calculations can lead to mistakes. A simple terminal-based application is needed to save basic book details in one file and calculate a fine from a user-entered number of late days.

The system should:

- Accept a book ID, title, and author when adding a book.
- Display all saved book records.
- Search for a book using its ID.
- Prevent duplicate book IDs.
- Calculate a fine using the number of late days entered by the user.
- Save book records in a plain-text file.
- Show clear messages for invalid choices or missing book ID.

3. OBJECTIVE

- Understand basic Linux terminal commands and Bash scripting.
- Create a menu-driven command-line application.
- Use variables and read to accept user input.
- Use case statements, loops, and if-else conditions.
- Read from and write to a plain-text file.
- Search records using grep and extract fields using cut.
- Calculate a simple fine using arithmetic expansion.

- Test normal operations and invalid input.

4. SCOPE OF THE PROJECT

This is a beginner-level academic project for a small local collection of books. It runs in Ubuntu Linux or Ubuntu through Windows Subsystem for Linux (WSL). It provides a terminal interface and stores records in books.txt. The fine calculator uses a fixed rate of ₹5 per late day and requires the user to enter the number of late days manually.

- The project does not issue or return books in this simplified version.
- It does not save borrower information or issue/return dates.
- It does not automatically calculate overdue days from dates.
- It is intended for single-user demonstration, not a large production library

5. EXISTING SYSTEM

In a manual system, book details may be recorded in a notebook or separate files. Searching for a record and calculating a fine may require extra time.

Limitations:

- Manual searching can be slow.
- Fine calculations may be performed inconsistently.
- Records may be difficult to organize.
- The process does not demonstrate command-line automation.

6. PROPOSED SYSTEM

The proposed system uses a Bash script and a text file. A menu lets the user select Add Book, Display Books, Search Book, Calculate Fine, or Exit. Each book record contains an ID, title, author, and status. The fine calculator multiplies late days by ₹5.

Basic workflow:

1. Start the Bash script.
2. Open or create books.txt.
3. Display the main menu.
4. Read the user's choice.
5. Perform the selected operation.
6. Save new book records when a book is added.
7. Return to the menu until Exit is selected.

7. REQUIREMENTS

7.1 Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM recommended.
- Keyboard and display.

7.2 Software Requirements

- Ubuntu Linux or Ubuntu through Windows Subsystem for Linux (WSL).
- Bash shell.
- Terminal application.
- Any text editor such as nano or vim.
- Standard utilities such as grep and cut.

8. SYSTEM DESIGN

8.1 Input

- Menu choice.
- Book ID, title, and author for adding a book.
- Book ID for searching.
- Number of late days for fine calculation.

8.2 Processing

- Check whether the selected menu option is valid.
- Check for an existing book ID before adding a record.
- Search books.txt for a matching ID.
- Calculate fine = late days × ₹5.
- Display a message when input is invalid or a book is not found.

8.3 Output

- Confirmation when a book is added.
- List of saved books.
- Search result for a book ID.
- Fine amount in rupees.
- Error message for invalid input.

8.4 Book Record Format

Field Name	Data Type	Example Value
BOOK_ID	String / Text	B101

Field Name	Data Type	Example Value
TITLE	String / Text	Linux Basics
AUTHOR	String / Text	A. Kumar
STATUS	String / Text	Available

9. ALGORITHM

- Start.
- Set the data file name to books.txt.
- Display the menu: Add Book, Display Books, Search Book, Calculate Fine, Exit.
- Read the user's choice.
- If Add Book is selected, read the ID, title, and author; reject a duplicate ID; then save the record.
- If Display Books is selected, show all saved records or a message if none exist.
- If Search Book is selected, find and display the record matching the entered ID.
- If Calculate Fine is selected, read the number of late days and validate that it is a non-negative whole number.
- Calculate the fine by multiplying late days by ₹5 and display the result.
- If Exit is selected, stop the program; otherwise return to the menu.
- End.

10. IMPLEMENTATION

10.1 Create the Project Directory

```
mkdir library_management_project
cd library_management_project
```

10.2 Create the Bash File

```
nano library.sh
```

10.3 Give Execute Permission

```
chmod +x library.sh
```

10.4 Run the Project

```
./library.sh
```

11. SOURCE CODE

```
#!/bin/bash

FILE="books.txt"
touch "$FILE"

while true
do
    echo ""
    echo "===== LIBRARY MANAGEMENT SYSTEM ====="
    echo "1. Add Book"
    echo "2. Display Books"
    echo "3. Search Book"
    echo "4. Calculate Fine"
    echo "5. Exit"

    read -p "Enter your choice: " choice

    case $choice in
      1)
         read -p "Enter Book ID: " id
         read -p "Enter Book Name: " name
         read -p "Enter Author Name: " author

         if [[ -z "$id" || -z "$name" || -z "$author" ]]; then
                echo "Book details cannot be empty!"
         elif [[ "$id$name$author" == *"|"* ]]; then echo "The pipe
                                                          character | is not allowed."


         elif grep -q "^$id|" "$FILE"; then echo "Book ID already exists!"

         else
                echo "$id|$name|$author|Available" >> "$FILE"
                echo "Book added successfully!"
            fi
            ;;

      2)
          if [[ -s "$FILE" ]]; then
                echo "===== BOOK DETAILS ====="
         printf "%-12s | %-25s | %-20s | %s\n" "BOOK ID" "BOOK NAME" "AUTHOR" "STATUS"
                echo "-------------------------------------------------------------------"
                while IFS='|' read -r id name author status
                do
          printf "%-12s | %-25s | %-20s | %s\n" "$id" "$name" "$author" "$status"
                done < "$FILE"
            else
                echo "No books found!"
            fi
            ;;

        3)
            read -p "Enter Book ID to search: " id
            if grep -q "^$id|" "$FILE"; then
                grep "^$id|" "$FILE" |
                while IFS='|' read -r bid name author status
                do
                    echo "Book ID: $bid"
                    echo "Book Name: $name"
                    echo "Author: $author"
                    echo "Status: $status"
                done
            else
                echo "Book not found!"
            fi
            ;;

        4)
            read -p "Enter number of late days: " days
            if [[ "$days" =~ ^[0-9]+$ ]]; then
```

```
                fine=$((days * 5))
                echo "Late Days: $days"
                echo "Fine Rate: Rs. 5 per day"
                echo "Fine Amount: Rs. $fine"
            else
                echo "Invalid input! Enter a non-negative whole number."
            fi
            ;;

        5)
            echo "Thank you for using the Library Management System!"
            break
            ;;

        *)
            echo "Invalid choice! Please select 1 to 5."
            ;;
    esac
done
```

12. SAMPLE EXECUTION

12.1 Successful Execution

```
Bash
$ chmod +x library_system.sh
$ ./library_system.sh


========================================
     LIBRARY MANAGEMENT SYSTEM
========================================
1. Add Book
2. Display Books
3. Search Book
4. Calculate Fine
5. Exit
Enter your choice: 1


Enter Book ID: B101
Enter Book Name: Linux Basics
Enter Author Name: A. Kumar
Book added successfully!
```

12.2 Invalid Input Example

```
Enter number of late days: -3
Invalid input! Enter a non-negative whole number.
Enter number of late days: 5
```

13. TESTING

Testing was performed using different input values to verify book addition, searching, fine calculation, and input validation.

Test	Input	Expected Result
1	Add book: 101, Java, James	Book added successfully
2	Add book with ID 101 again	Book ID already exists
3	Display books	Book details displayed
4	Search Book ID: 101	Book details displayed
5	Late days: 5	Fine: Rs. 25
6	Late days: -3	Invalid input
7	Search Book ID: 999	Book not found
8	Empty book details	Book details cannot be empty

14. TEAM MEMBER RESPONSIBILITIES

Team Member	Responsibilities	Deliverables
SAHIL KUMAR	Problem analysis, Bash programming, book management logic, fine calculation and input validation	Source code and algorithm
GUNTI SAI KARTHIK	Testing, sample outputs, documentation and presentation	Test cases and project report

15. ADVANTAGES AND LIMITATIONS

15.1 Advantages

- Simple and suitable for beginners.
- Runs directly in the Linux terminal.
- Stores book records in a local text file.
- Supports adding, displaying, and searching books.
- Calculates a simple fine quickly.
- Demonstrates Bash loops, conditions, and file handling.

15.2 Limitations

- No graphical user interface or login system.
- Does not issue or return books in this version.
- Does not store borrower details or due dates.
- Fine is based on manually entered late days.
- Plain-text storage is not suitable for large or concurrent collections.
- The pipe character is used as a field separator.

16. FUTURE ENHANCEMENTS

- Add issue and return operations.
- Store borrower details and due dates.
- Calculate overdue days automatically from dates.
- Add update and delete book options.
- Use CSV or a database for more reliable storage.
- Add librarian login and permissions.
- Generate reports of book records and collected fines.
- Add backups and stronger input validation.

17. LEARNING OUTCOMES

After completing this project, students should be able to:

- Navigate directories using pwd, ls, and cd.
- Create folders and files using mkdir, touch, and nano.
- Change file permissions using chmod.
- Write and execute a Bash script.
- Use variables and read for input.

- Use case, loops, and if statements.
- Read and update plain-text files.
- Use grep and file redirection for basic data operations.
- Perform arithmetic calculations and validate numeric input.
- Test program behavior and troubleshoot errors.

18. CONCLUSION

The Library Management System demonstrates how Bash shell scripting can automate basic book-record tasks. The program provides options to add, display, and search books, and to calculate a fine from manually entered late days. It brings together variables, loops, conditional statements, file handling, text searching, and arithmetic operations. It is a useful beginner-level project and can be extended with issue/return tracking, automatic overdue calculation, borrower details, and database storage.

19. REFERENCES

- GNU Bash Reference Manual.
- Ubuntu Linux documentation.
- Linux manual pages for bash, chmod, mkdir, grep, touch, and related commands.
- Course notes and laboratory material provided by the faculty.

APPENDIX A – BASIC COMMANDS USED

Command	Purpose
pwd	Display the current working directory.
ls	List files and directories.
cd	Change directory.
mkdir	Create a project directory.
nano	Create or edit a text file
touch books.txt	Create an empty data file if it does not exist.
chmod +x library.sh	Give the script execute permission.
`./library.sh`	Run the Bash script.
`cat books.txt`	Display saved book records.
`grep`	Search for matching text in a file.

APPENDIX B – VIVA QUESTIONS

No.	Question
1	What is Linux?
2	What is Bash scripting?
3	What is the purpose of the Library Management System?
4	Why is chmod +x used?
5	What is the purpose of the read command?
6	How does the program add a new book?
7	How does the program display all books?
8	How does the program search for a book by ID?
9	How is the library fine calculated?
10	Why is books.txt used in this project?
11	What is the purpose of the grep command?
12	How does the program prevent duplicate book IDs?
13	What is the purpose of the case statement?
14	How does the program validate user input?
15	How can the project be extended to include book issuing and returning?

```
#!/bin/bash

FILE="books.txt"
touch "$FILE"

while true
do
    echo ""
    echo "===== LIBRARY MANAGEMENT SYSTEM ====="
    echo "1. Add Book"
    echo "2. Display Books"
    echo "3. Search Book"
    echo "4. Calculate Fine"
    echo "5. Exit"

    read -p "Enter your choice: " choice

    case $choice in
      1)
         read -p "Enter Book ID: " id
         read -p "Enter Book Name: " name
         read -p "Enter Author Name: " author

         if [[ -z "$id" || -z "$name" || -z "$author" ]]; then
                echo "Book details cannot be empty!"
         elif [[ "$id$name$author" == *"|"* ]]; then echo "The pipe
                                                          character | is not allowed."

         elif grep -q "^$id|" "$FILE"; then echo "Book ID already exists!"
```

```
         else
                echo "$id|$name|$author|Available" >> "$FILE"
                echo "Book added successfully!"
            fi
            ;;

      2)
            if [[ -s "$FILE" ]]; then
                echo "===== BOOK DETAILS ====="
            printf "%-12s | %-25s | %-20s | %s\n" "BOOK ID" "BOOK NAME" "AUTHOR" "STATUS"
                echo "-------------------------------------------------------------"
                while IFS='|' read -r id name author status
                do
             printf "%-12s | %-25s | %-20s | %s\n" "$id" "$name" "$author" "$status"
                done < "$FILE"
            else
                echo "No books found!"
            fi
            ;;

        3)
            read -p "Enter Book ID to search: " id
            if grep -q "^$id|" "$FILE"; then
                grep "^$id|" "$FILE" |
                while IFS='|' read -r bid name author status
                do
                    echo "Book ID: $bid"
                    echo "Book Name: $name"
                    echo "Author: $author"
                    echo "Status: $status"
                done
```

```
            else
                echo "Book not found!"
            fi
            ;;

        4)
            read -p "Enter number of late days: " days
            if [[ "$days" =~ ^[0-9]+$ ]]; then
                fine=$((days * 5))
                echo "Late Days: $days"
                echo "Fine Rate: Rs. 5 per day"
                echo "Fine Amount: Rs. $fine"
            else
                echo "Invalid input! Enter a non-negative whole number."
            fi
            ;;

        5)
            echo "Thank you for using the Library Management System!"
            break
            ;;

        *)
            echo "Invalid choice! Please select 1 to 5."
            ;;
    esac
done
```

```
sahil@HP:~$ nano library.sh
sahil@HP:~$ ./library.sh

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Display Books
3. Search Book
4. Calculate Fine
5. Exit
Enter your choice: 1
Enter Book ID: B101
Enter Book Name: Linux Basics
Enter Author Name: A. Kumar
Book added successfully!
```

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Display Books
3. Search Book
4. Calculate Fine
5. Exit
Enter your choice: 2
===== BOOK DETAILS =====
BOOK ID      | BOOK NAME                | AUTHOR              | STATUS
-------------------------------------------------------------
B101         | Linux Basics             | A. Kumar            | Available
```

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Display Books
3. Search Book
4. Calculate Fine
5. Exit
Enter your choice: 3
Enter Book ID to search: B101
Book ID: B101
Book Name: Linux Basics
Author: A. Kumar
Status: Available
```

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Display Books
3. Search Book
4. Calculate Fine
5. Exit
Enter your choice: 4
Enter number of late days: 5
Late Days: 5
Fine Rate: Rs. 5 per day
Fine Amount: Rs. 25
```

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Display Books
3. Search Book
4. Calculate Fine
5. Exit
Enter your choice: 5
Thank you for using the Library Management System!
sahil@HP:~$
```